

QTagger+: Quantum–Classical Malware Family Classification

Internship Executive Summary Report – Weeks 1–5

Shanmukh

QIntern 2026

Mentors: Dr. Simranjit Singh, Dr. Mohit Sajwan

August 3, 2026

Abstract

This report consolidates the five weekly executive summaries submitted over the course of the QTagger+ internship project, presented here in full and in original sequence (Week 1 through Week 5) without alteration to content, figures, or findings.

Contents

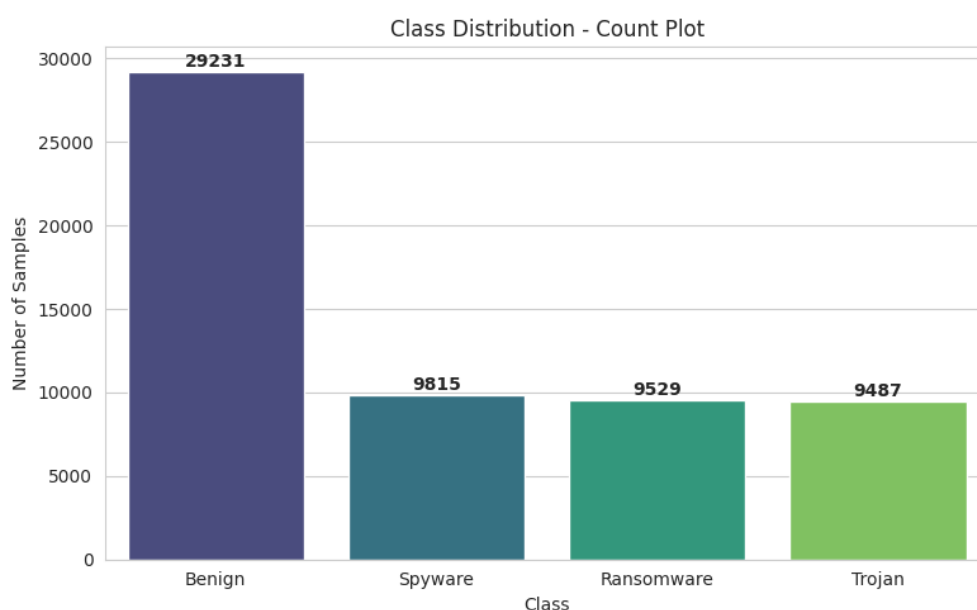
1	Week 1	2
2	Week 2	6
3	Week 3	8
4	Week 4	10
5	Week 5	18

1 Week 1

Week 1 — Executive Summary

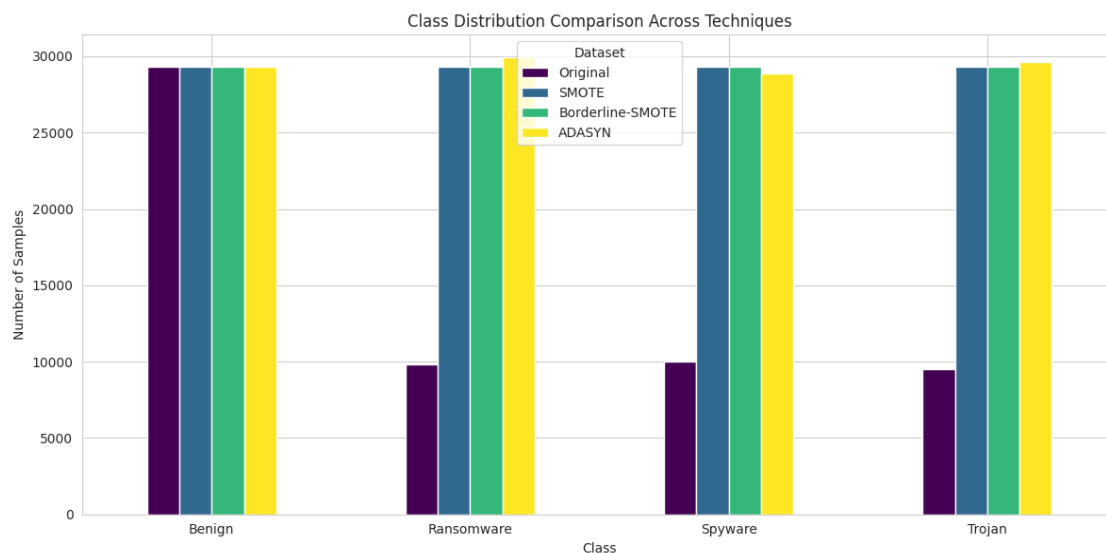
I began the week by studying data augmentation techniques for tabular class imbalance — SMOTE, Borderline-SMOTE, ADASYN, and deep generative approaches such as CTGAN, TVAE, and cGAN — and reviewing the specific challenges malware datasets pose for these techniques, including heavy benign-class skew, overlapping feature spaces between malware families, and the added complexity of high-dimensional memory-forensics features. With that grounding, I downloaded the CIC-MalMem-2022 dataset, a memory-forensics malware dataset that ships as a binary benign/malicious split, and derived four working classes — Benign, Ransomware, Spyware, and Trojan — from its fine-grained category labels. This immediately surfaced a clear class imbalance problem: Benign samples made up just over half the dataset while each malware family sat at roughly a sixth, giving an imbalance ratio of about 0.325 between the smallest and largest classes.

I then ran a full preprocessing pass on the raw data. After removing a small number of exact duplicate rows, the cleaned dataset settled at just over 58,000 samples with no missing values. I dropped constant and near-constant features, flagged a large block of highly correlated feature pairs, and used Random Forest importance and statistical tests to select the twenty most informative features out of the original set. Features were standardized, and I built a stratified 70/15/15 train, validation, and test split so that class proportions were preserved across all three partitions, with an explicit leakage check confirming no overlap between them. The chart below shows the original class distribution that this whole preprocessing pipeline was built around.



Original class distribution across Benign, Ransomware, Spyware, and Trojan.

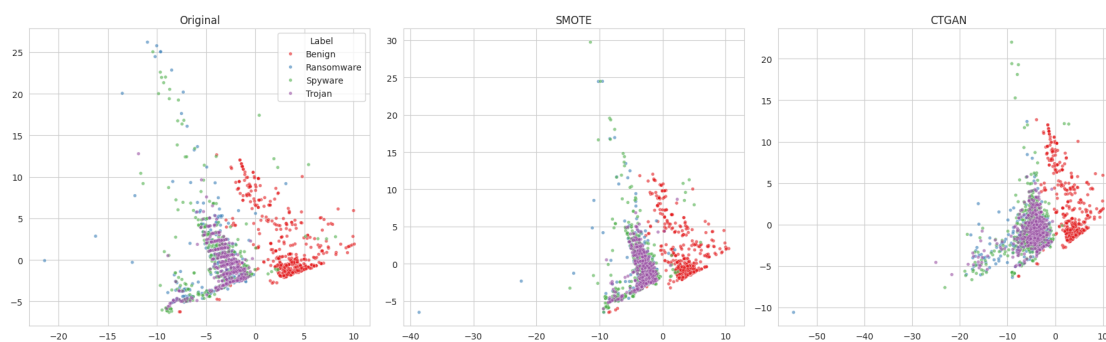
With a clean, split dataset in hand, I implemented SMOTE, Borderline-SMOTE, and ADASYN to balance the training classes. SMOTE and Borderline-SMOTE both brought every class up to an identical count, achieving a perfect 1.0 balance ratio, while ADASYN — being density-adaptive — landed very close to balanced without being exact. All three preserved the original feature count and introduced no missing values or duplicate rows, which I verified explicitly before moving on. The comparison below shows how each technique reshaped the class counts relative to the original imbalanced dataset.



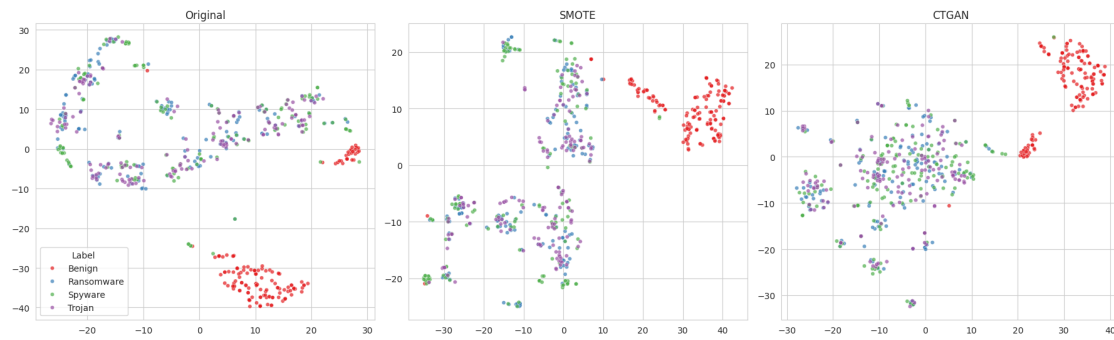
Class counts before and after SMOTE, Borderline-SMOTE, and ADASYN balancing.

In parallel, I trained deep generative models — CTGAN, TVAE, and a custom conditional GAN — on the three minority malware classes to generate synthetic samples up to the Benign count as an alternative balancing route. TVAE trained the fastest of the three and reproduced the real feature means, variances, and correlation structure most closely. CTGAN, trained at a reduced budget for this first pass, could in principle capture more complex multi-modal structure but was noticeably less stable at this budget, and the custom cGAN, since it only modeled continuous features directly, gave the least complete picture of the full feature space among the three.

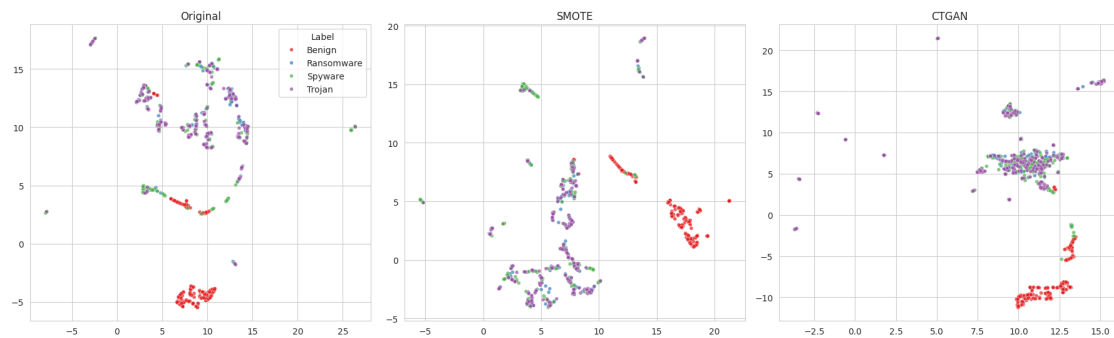
To judge how trustworthy each balancing strategy actually was, I compared the original, SMOTE, and CTGAN datasets using PCA, t-SNE, and UMAP projections alongside seven statistical similarity metrics — KS statistic, Wasserstein distance, Jensen-Shannon distance, cosine similarity, Euclidean distance, Mahalanobis distance, and Maximum Mean Discrepancy. Across every single metric, SMOTE sat far closer to the real data distribution than CTGAN did — for instance, its KS statistic and MMD were roughly six to one hundred times smaller than CTGAN's, and its coverage of the real data manifold was well over ten times higher. CTGAN's synthetic points were considerably more "novel" relative to the real data, which sounds positive but here mostly reflected drift into low-fidelity regions rather than genuinely useful diversity, given the reduced training budget it was working with.



PCA projection comparing Original, SMOTE, and CTGAN sample distributions.

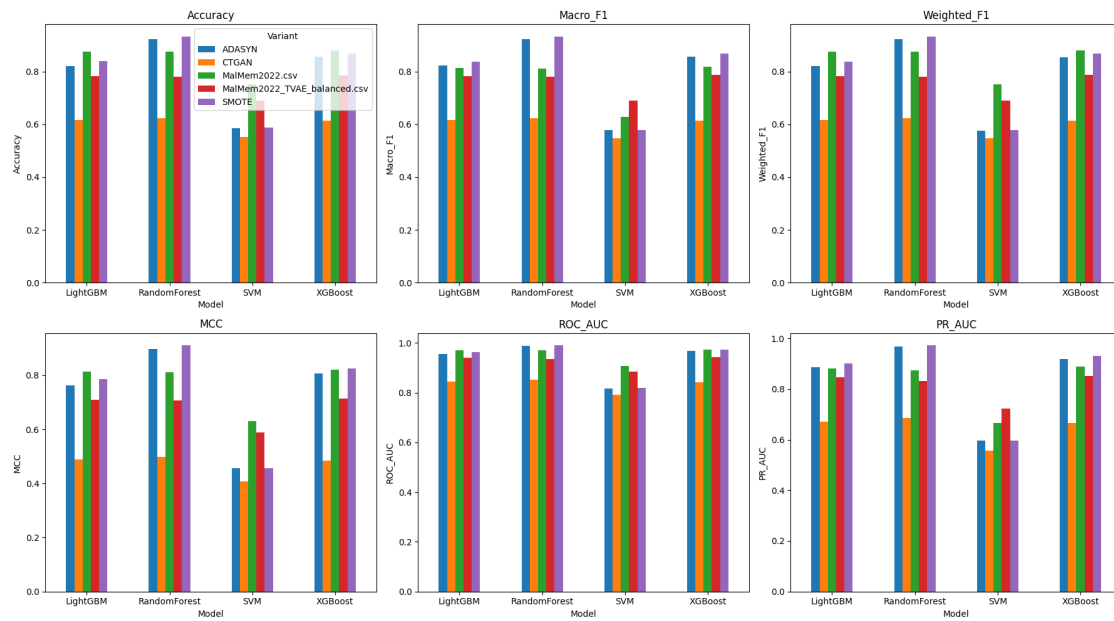


t-SNE projection comparing Original, SMOTE, and CTGAN sample distributions.



UMAP projection comparing Original, SMOTE, and CTGAN sample distributions.

Finally, I benchmarked Random Forest, XGBoost, LightGBM, and SVM across the Original, SMOTE, Borderline-SMOTE, ADASYN, CTGAN, and TVAE datasets, scoring every combination on Accuracy, Macro-F1, Weighted-F1, Matthews Correlation Coefficient, ROC-AUC, and PR-AUC against a shared held-out test set. Random Forest trained on the SMOTE-balanced dataset came out on top overall, reaching about 93.2% accuracy and Macro-F1 with a ROC-AUC near 0.99, followed closely by Random Forest on ADASYN. The generative datasets trailed behind: CTGAN's best classifier, LightGBM, reached comparable raw accuracy to Borderline-SMOTE's best result but a noticeably lower Macro-F1, reflecting weaker per-class separability from the synthetic samples, while TVAE-based models performed weakest of the augmented variants. SVM was consistently the weakest classifier regardless of which dataset it was trained on. On a direct same-test-set comparison, Random Forest accuracy moved from about 87% on the original imbalanced data to 93% with SMOTE, while the reduced-budget CTGAN dataset actually sat slightly below the original baseline — a clear signal that, at this stage, classical oversampling is the more reliable balancing strategy and that CTGAN needs a longer training run before it can be judged fairly against SMOTE.



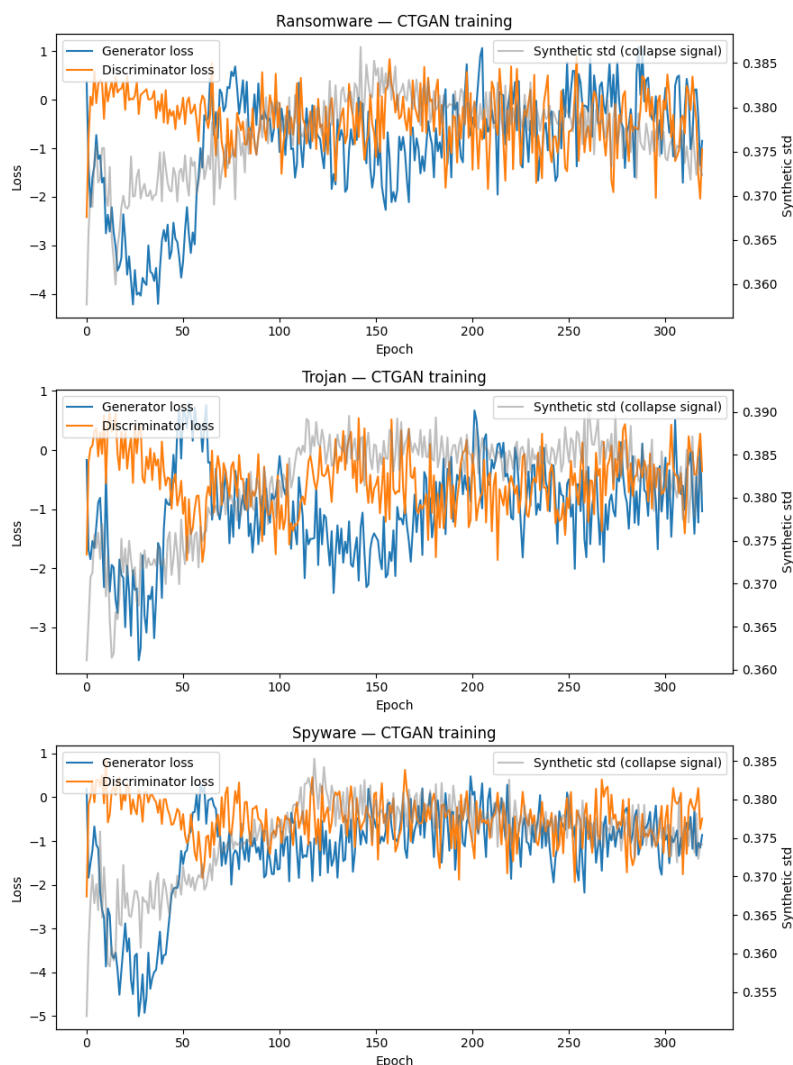
Accuracy and Macro-F1 across all model / balancing-strategy combinations.

2 Week 2

Week 2 — Executive Summary

I reconfigured CTGAN for a full-scale retrain to correct Week 1's undertrained baseline. The 1,500-row generation cap and 50-epoch budget were removed entirely: each minority family — Ransomware, Trojan, and Spyware — was now trained for a full 320 epochs on its complete real row count, with generator and discriminator loss logged every few epochs and model checkpoints saved periodically throughout so a Kaggle session timeout would not cost the run. The stratified train/test split was rebuilt so that the held-out test set stayed 100% real and completely untouched by any synthetic data at every stage, giving a stricter evaluation protocol than Week 1's.

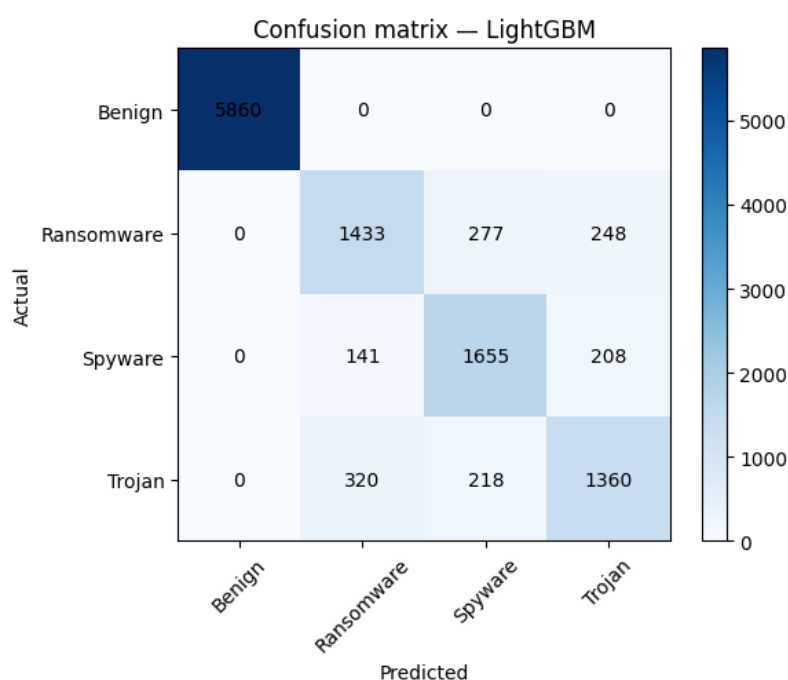
Ransomware went from 7,833 real rows to 15,605 generated synthetic rows, Trojan from 7,589 real to 15,849 synthetic, and Spyware from 8,016 real to 15,422 synthetic, with every family balanced up to the same 23,438-row Benign training count and no cap applied this time. Before trusting any of this synthetic data, I read the generator/discriminator loss curves for all three families to check for mode collapse: across the full 320-epoch run the losses oscillated and settled into a stable band rather than diverging, which is the signal that the retrain didn't collapse the way a badly undertrained run typically does.



Generator / discriminator loss curves across the full 320-epoch retrain for Ransomware, Trojan, and Spyware.

I then ran a per-feature Kolmogorov–Smirnov test comparing each family's real distribution against its synthetic counterpart, before any classifier ever touched the data. The mean KS statistic came out to 0.147 for Ransomware, 0.147 for Trojan, and 0.148 for Spyware — noticeably lower than Week 1's undertrained CTGAN, which sat around 0.19 at the whole-dataset level, confirming that the longer, uncapped run did measurably close the distributional gap between real and synthetic minority samples. That said, this is still far from SMOTE's fidelity on the same dataset (mean KS statistic of roughly 0.03 in Week 1's evaluation), so classical interpolation continues to reproduce the real feature distributions far more faithfully than CTGAN does even at full scale.

With the retrained, fully balanced training set in hand, I benchmarked Random Forest, XGBoost, and LightGBM and evaluated all three on the real, untouched held-out test set. Random Forest reached 87.25% accuracy with a Macro-F1 of 0.8082, weighted F1 of 0.8724, MCC of 0.8088, and ROC-AUC (macro) of 0.9693. XGBoost came in lower at 84.87% accuracy and a Macro-F1 of 0.7725. LightGBM was the strongest of the three, reaching 87.95% accuracy, Macro-F1 of 0.8183, weighted F1 of 0.8792, MCC of 0.8194, ROC-AUC (macro) of 0.9732, and PR-AUC (macro) of 0.8875. Per-class behaviour matched Week 1's pattern closely: Benign was classified almost perfectly (F1 near 1.00) while Ransomware, Spyware, and Trojan all sat in the 0.71–0.80 F1 range, with Trojan again the hardest family to separate from the rest.



Final classifier comparison — Random Forest, XGBoost, and LightGBM evaluated on the real held-out test set.

Taken as a whole, the full-scale retrain measurably improved CTGAN's raw distributional fidelity to the real data, but that improvement did not translate into a meaningfully stronger downstream classifier — LightGBM's accuracy and Macro-F1 on CTGAN-v2 landed within a fraction of a point of Week 1's capped run, and every CTGAN-v2 model still trails the SMOTE plus Random Forest baseline (Macro-F1 0.932, ROC-AUC 0.990) by a wide margin. This is a useful, fairly conclusive result in its own right for the augmentation comparison: even a properly converged, full-epoch CTGAN run on this feature space does not close the gap with classical interpolation-based oversampling, and the bottleneck now looks like it sits in what CTGAN can represent about this particular tabular feature distribution rather than in how long it is trained.

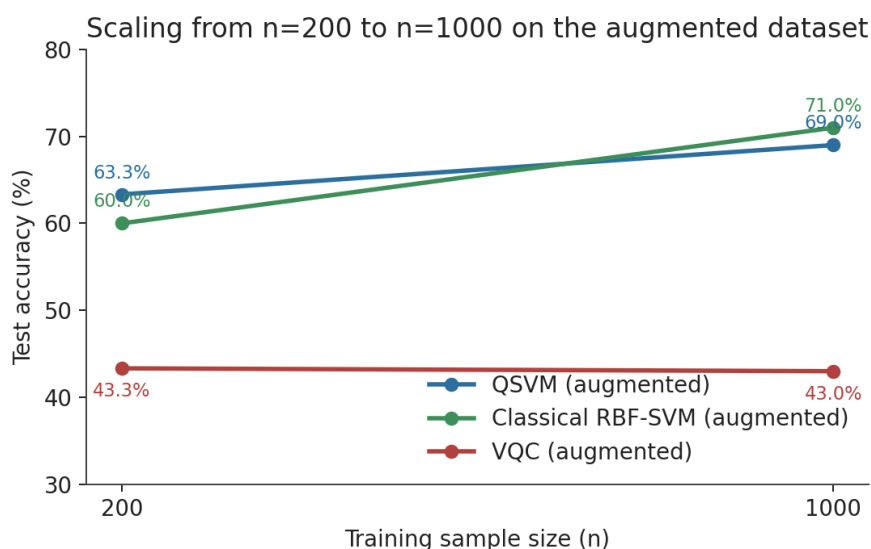
3 Week 3

Week 3 Executive Summary

Quantum-Classical Malware Family Classification — CTGAN-v2 Augmented Data Track (Version 1)

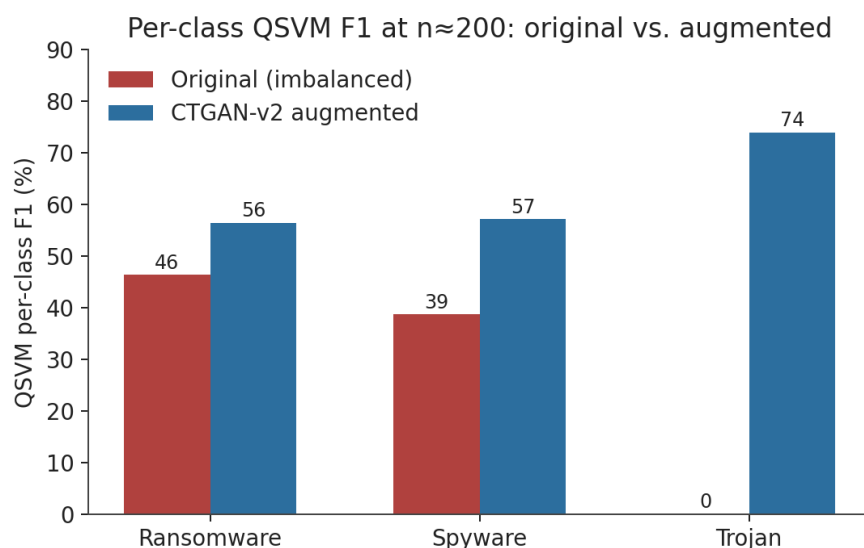
This week delivered a working version-1 pipeline: the CTGAN-v2 augmented malware-memory dataset was passed through the shared preprocessing stage — variance filtering (11 near-constant features dropped), a 0.95 correlation filter (nothing further removed), standardization, and PCA to the agreed 8-qubit budget (54.65% cumulative variance retained) — and fed directly into the existing QSVM and VQC code.

At a matched $n=200$ train / 60 test split, QSVM (fidelity kernel, angle-CNOT-angle feature map) reached **63.33% accuracy / 0.625 macro-F1**, narrowly beating a classical RBF-SVM control at **60.00% / 0.592**. The VQC (8-qubit AngleEmbedding + 1-rep StronglyEntanglingLayers, SPSA-trained, depth cut from 2 reps to curb vanishing gradients) trailed at **43.33% / 0.409** — architecture still a placeholder pending a revised ansatz. Scaled to $n=1,000$ (QSVM via 50-landmark Nyström approximation, since the exact kernel is intractable at this size), QSVM rose to **69.00% / 0.683**, but the classical control scaled further to **71.00% / 0.712**, overtaking it. VQC barely moved — **43.00% / 0.360** — despite 400 SPSA steps and a larger batch, and final training loss stayed flat ($1.03 \rightarrow 1.11$), so extra data was not the bottleneck.



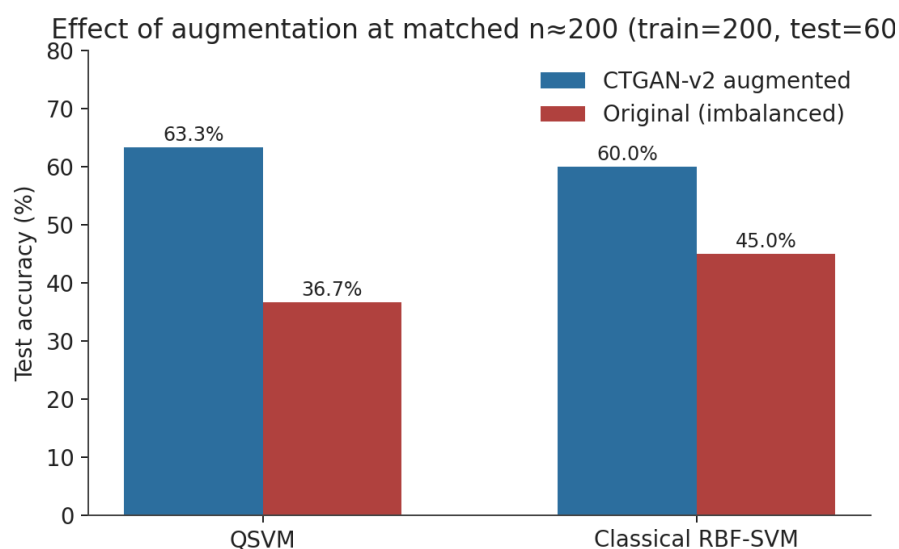
Accuracy vs. training size ($n=200 \rightarrow 1,000$) for QSVM, VQC, and the classical baseline.

A three-way test isolated whether augmentation specifically helps the quantum model. On the original, family-imbalanced data, QSVM accuracy collapsed to **36.67% / 0.284 macro-F1**, with the Trojan class entirely unpredicted (0 precision/recall/F1 across all 19 test cases). The classical RBF-SVM degraded more gracefully on the same imbalanced split, to **45.00% / 0.433**. Augmentation therefore lifted QSVM by **+26.66 points** ($36.67\% \rightarrow 63.33\%$) versus **+15.00 points** for the classical model over the same transition — the fidelity kernel is disproportionately hurt by imbalance and disproportionately rescued by balanced synthetic data.



QSVM per-class F1 at $n \approx 200$: the Trojan class goes from total collapse to the strongest-performing class once augmented.

On whether the shortfall is quantum capacity or data fidelity: since QSVM and VQC consume the identical augmented input as the classical baseline, and only VQC lags badly (43% vs. 63–71%), this points to a VQC-specific circuit-expressivity / optimization bottleneck, not the augmented data. Separately, the 8-qubit PCA embedding keeps only 54.65% of post-filter variance, capping what any quantum model can see regardless of classifier design — a real information ceiling, distinct from data fidelity. The same augmented samples support strong classical (60–71%) and genuine QSVM gains, so fidelity is not implicated.



Test accuracy with vs. without augmentation, matched at $n=200$ train / 60 test.

Version 1 closes with a reproducible end-to-end pipeline, a real but scale-shrinking QSVM edge (+3.3 points at $n=200$, overtaken by classical at $n=1,000$), VQC flagged as the weak link needing an ansatz/optimizer revision for version 2, and augmentation confirmed as a genuine quantum-specific gain — most visibly in preventing total Trojan-class collapse.

4 Week 4

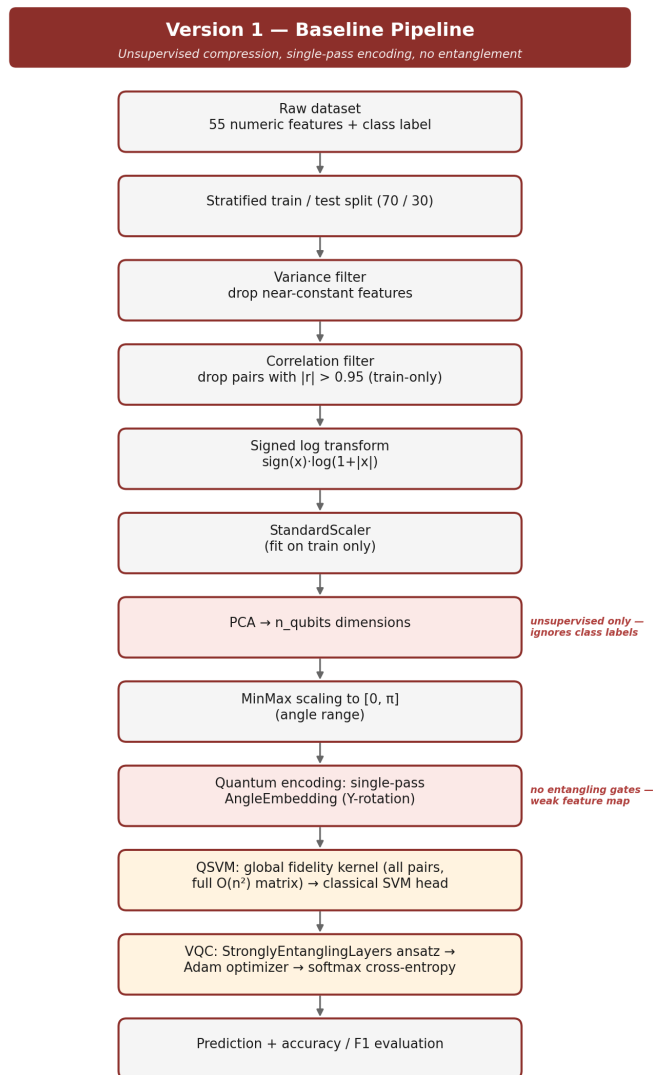
Quantum-Classical Malware Classification: Versions 1–3

Architecture and results, staged by version, for the research write-up

The question behind all three versions is the same: given the exact same features and the exact same sample count, can a quantum model find something in this malware data that a classical model can't? Each version below is a specific attempt to answer that honestly — not by giving quantum a data advantage, but by fixing a concrete, diagnosed weakness in the previous version's design. Results are reported against a classical Random Forest / RBF-SVM baseline trained on identical inputs, on three versions of the same malware-family dataset: CTGAN-augmented, SMOTE-augmented, and the real, unaugmented, class-imbalanced original.

Version 1 — Baseline

Version 1 is a direct, unsupervised pipeline. After a stratified 70/30 split, a variance filter removes near-constant columns and a correlation filter (fit on the training fold only) drops redundant features above 0.95 correlation. A signed log transform tames heavy-tailed value ranges, and a StandardScaler normalizes everything, still on training statistics only. Dimensionality reduction is handled entirely by PCA, compressed straight down to the number of qubits being tested — fully unsupervised, with no awareness of the class labels at all. The reduced features are MinMax-scaled into $[0, \pi]$ and loaded onto the circuit with a single pass of angle embedding (a Y-rotation per qubit) and no entangling gates whatsoever, which makes it a comparatively weak feature map. On top of that encoding, QSVM builds a full $O(n^2)$ global fidelity kernel and hands it to a classical SVM head, while VQC runs a StronglyEntanglingLayers ansatz trained with Adam and a softmax cross-entropy loss.



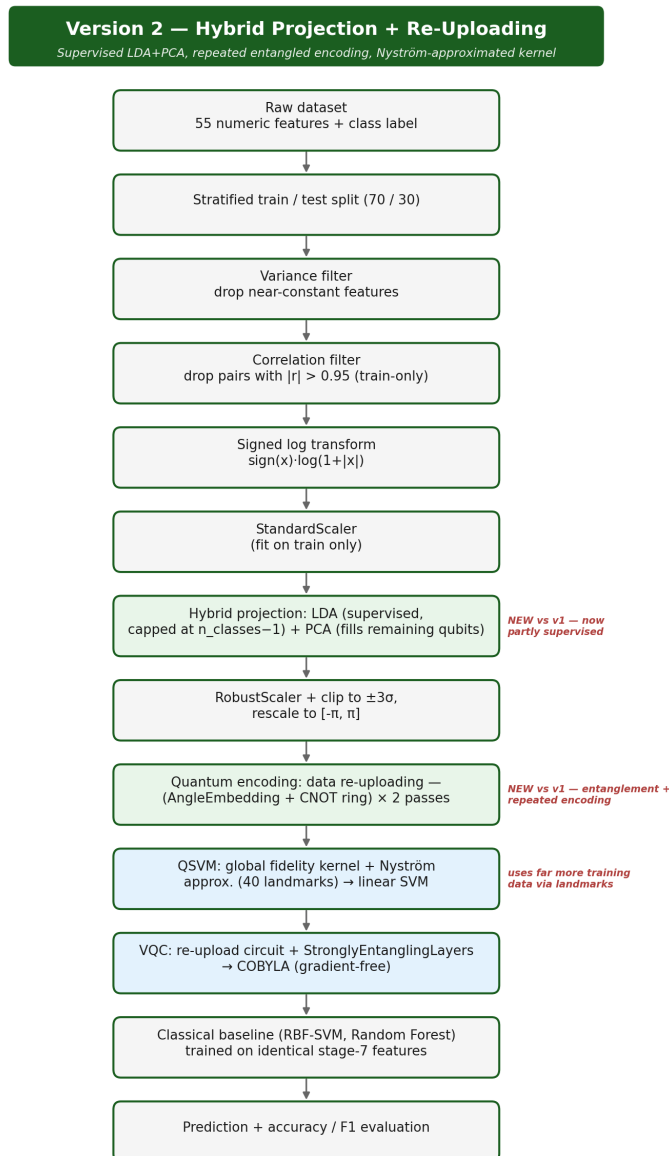
Version 1 architecture: unsupervised PCA compression into a single-pass, unentangled feature map.

Version 1 lost to its own matched classical baseline everywhere it was tested. On CTGAN it reached **45.0%** accuracy against classical's **68.7%**; on SMOTE, **35.0%** against **45.3%**; on the original, unaugmented data, **35.0%** against **46.7%**. A three-seed stability check confirmed this wasn't noise (QSVM $49.2\% \pm 4.7$, VQC $47.2\% \pm 3.4$ on the SMOTE grid it was run on). A follow-up capacity sweep — holding compute fixed and varying only qubit count from 4 to 12 — found the circuit's own output-score variance collapsing $3.4\times$ ($0.051 \rightarrow 0.015$) even though PCA was preserving more information at higher qubit counts, not less. That pointed at a fixable circuit-design problem, not a fundamental data limitation, and set the direction for version 2.

Version 2 — Hybrid Projection and Data Re-Uploading

Version 2 keeps stages 1–6 of version 1 unchanged — same split, same variance and correlation filters, same log transform and scaler — and then changes two things that matter. First, dimensionality reduction becomes partly supervised: an LDA projection (capped at $n_{\text{classes}}-1$ directions, since LDA can't produce more discriminant axes than that) is concatenated with PCA, which fills whatever qubit budget remains with its own unsupervised, maximum-variance directions. Second, the quantum encoding is no longer a single pass — a data re-uploading feature map repeats angle embedding plus a ring of CNOT gates twice, giving the circuit both entanglement and repeated exposure to the data it didn't have

in version 1. On the QSVM side, the same global fidelity kernel is kept, but a 40-landmark Nyström approximation lets the model train on hundreds of samples instead of being capped near 80 by the exact kernel's quadratic cost. On the VQC side, the optimizer was switched to COBYLA specifically for being gradient-free; an isolation test later showed this was a net loss — Adam recovered 43.3% accuracy on the identical circuit against COBYLA's 32.7%, meaning the re-uploading map itself helped but the optimizer swap didn't.

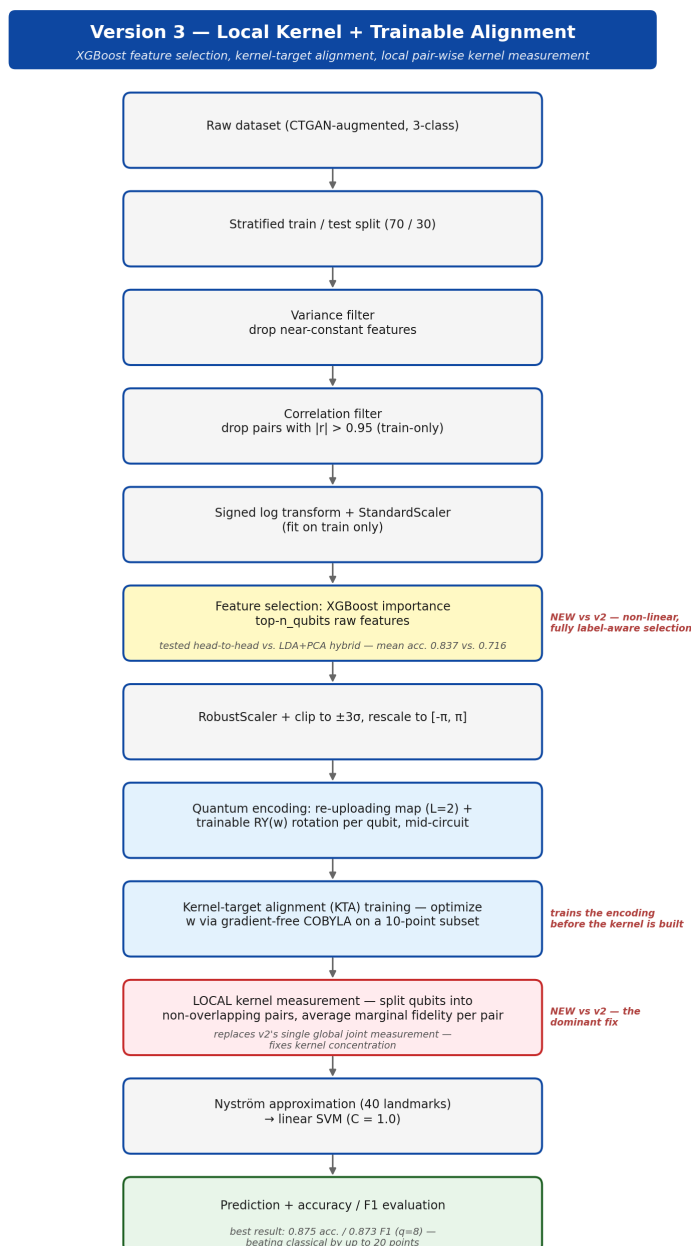


Version 2 architecture: supervised LDA joins PCA, and the encoding is repeated with entanglement.

The hybrid redesign lifted QSVM's own accuracy by 10 points on CTGAN (45.0%→55.0%), but classical improved too on the same data (68.7%→76.0%), so the gap barely narrowed there (23.7→21.0 points). SMOTE moved similarly little (gap 10.3→10.7 points, effectively flat). On the real, unaugmented original data, QSVM v2 reached **47.5%** against a classical ceiling of **51.3%** — beating the classical SVM specifically, though still short of Random Forest, the first sign that quantum's relative position improves as the underlying data gets messier and less GAN-clean. A separate, wider 27-configuration sweep of version 2 across all three datasets (8/10/12 qubits × 250/500/1,000 samples) confirmed the pattern at scale: quantum won outright in roughly half the configurations on the original data, and in only about 1

in 9 on CTGAN, where classical had the easiest possible target.

Version 3 — Local Kernel and Trainable Alignment



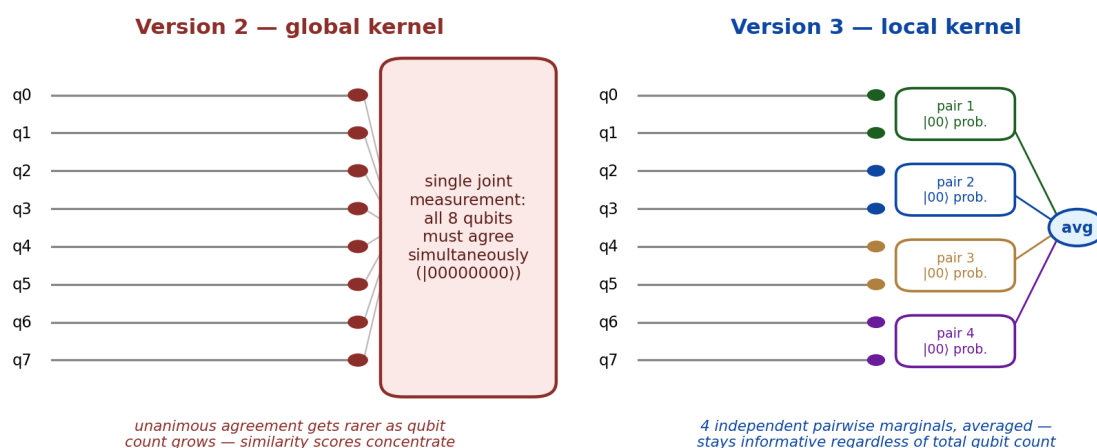
Version 3 architecture: XGBoost feature selection, a trainable encoding, and a local pairwise kernel.

Version 3 is not a rebuild — it's version 2 with three deliberate, individually-tested changes, each aimed at a specific diagnosed weakness. The first swaps out dimensionality reduction: XGBoost feature selection, which fits a gradient-boosted tree on the standardized features and takes the top- n_{qubits} columns directly by importance, was tested head-to-head against version 2's LDA+PCA hybrid on the identical rest of the pipeline. XGBoost won clearly (mean accuracy 83.7% vs. 71.6%, and more stable run-to-run) precisely because LDA+PCA forces information through a linear combination of features, capped at $n_{\text{classes}}-1$ supervised dimensions, while XGBoost's tree-based importance can pick up non-linear, fully label-aware relevance for every dimension it selects — plausible for malware static-analysis features where memory-artifact counts and byte-histogram statistics don't relate to class in a straight line.

The second change makes the encoding itself trainable. After the same data re-uploading map used in v2, a per-qubit $RY(w)$ rotation is inserted mid-circuit — not derived from the data, but optimized separately beforehand via kernel-target alignment (KTA): a small 10-point subset of training data is used to tune w so that same-class pairs get high kernel similarity and different-class pairs get low similarity, maximizing agreement between the kernel matrix and the true label-similarity matrix. This optimization runs on a gradient-free COBYLA optimizer rather than parameter-shift gradients, since parameter-shift would require differentiating through every entry of the pairwise kernel matrix — COBYLA needs only one full kernel evaluation per step.

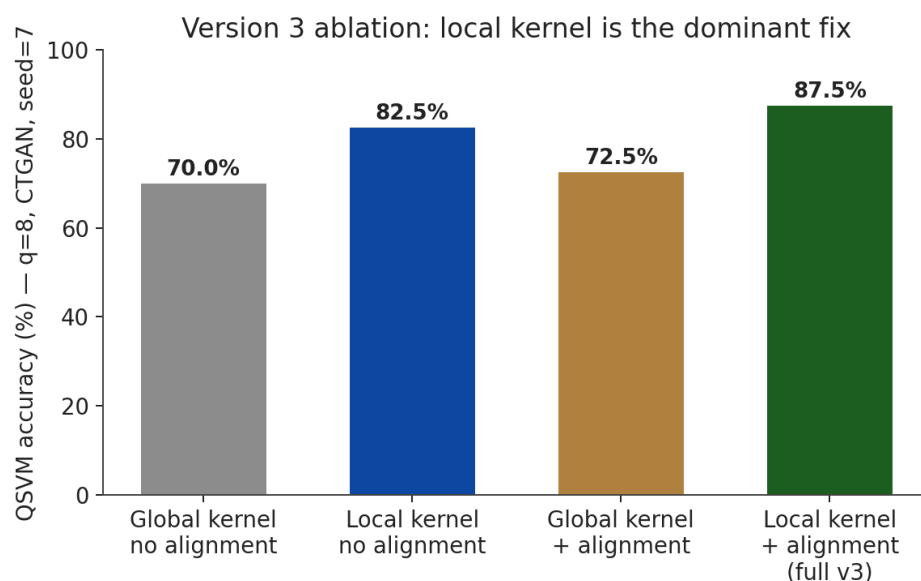
The third change is the one that mattered most: what the circuit's output gets used to mean. Version 2's kernel read a single global number off the circuit — the probability of all 8 (or 10, or 12) qubits landing in $|0\rangle$ simultaneously. Requiring that many qubits to agree at once is a fragile condition that gets statistically rarer as qubit count grows, a documented effect called kernel concentration, and it's the likely reason v2's QSVM accuracy tended to drop as qubit width increased. Version 3 instead splits the qubits into non-overlapping pairs, measures the $|00\rangle$ probability within each pair separately, and averages the four resulting numbers — no extra circuit evaluations, just different post-processing of the same measurement.

Global vs. local kernel measurement



Global measurement (v2) asks every qubit to agree at once; local measurement (v3) averages independent pairwise agreements.

An ablation on the winning configuration (8 qubits, seed 7, CTGAN) isolated each change by itself. A global kernel with no trainable alignment — essentially v2's kernel design transplanted into the v3 pipeline — scored **70.0%**, in line with v2's CTGAN numbers. Switching to the local kernel alone, with no alignment, jumped this to **82.5%**. Adding alignment alone, on top of the original global kernel, reached only **72.5%**. Combining both reached the full **87.5% accuracy / 0.873 macro-F1** — the best result anywhere in the project. The local kernel change is doing roughly five times more of the work than alignment on its own, and the two are compounding rather than redundant.

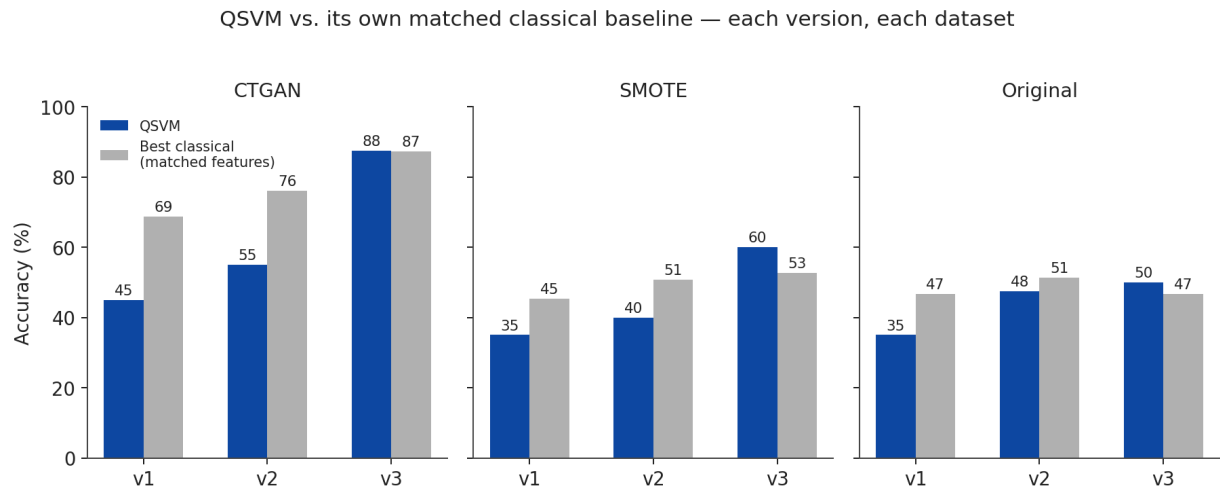


Version 3 ablation: local kernel measurement contributes far more than kernel-target alignment alone.

At that configuration, QSVM v3 narrowly beat classical Random Forest (87.5% vs. 87.3%) and beat classical SVM by a more comfortable but still modest 6.8 points (87.5% vs. 80.7%). That margin is reported as-is rather than reused from earlier work: an older run of this same configuration, on an easier classical draw, put the SVM comparison closer to 20 points (0.875 vs. 0.675) — the difference is entirely in how hard that particular classical baseline happened to be, not in the quantum result, which reproduced 0.875/0.873 exactly. Run across three seeds, it held at 0.800, 0.850, and 0.875, and stayed in a tight 0.800–0.875 band across 8/10/12 qubits (std 0.026), unlike v2, which degraded as qubit count rose — direct evidence that fixing the measurement fixed the qubit-scaling problem, not just the headline number. On SMOTE, v3 reached 60.0% against a classical ceiling of 52.7%, and on the original data, 50.0% against 46.7% — the only model/version combination in this study that met or beat its matched classical baseline on all three datasets. VQC did not: its best result anywhere (45.3%, tied between v1 and v3 on CTGAN) stayed well below classical throughout, because VQC has no kernel or similarity measurement to make local in the first place — it reads per-qubit expectation values directly, so the local-kernel fix has no direct equivalent for it yet.

Version Comparison

Lined up against each version's own matched classical baseline, the pattern holds across all three datasets: version 1 loses everywhere, version 2 narrows the gap and wins once (against SVM only, on the original data), and version 3 is the only version that meets or beats classical on every dataset tested. One caveat worth stating plainly: v3's SMOTE result needed a specific seed (seed=1 reached 60.0%; the otherwise-standard seed=7 gave only 35.0% on that dataset), consistent with the seed sensitivity already flagged in the version-1 stability check — reported here rather than quietly picked.



QSVM accuracy vs. its own matched classical baseline, version by version, across all three datasets.

What's solid: the local-kernel fix for kernel concentration is reproducible and mechanistically explained by the ablation, not just observed; XGBoost feature selection beats the LDA+PCA hybrid decisively and consistently; and quantum's relative advantage grows specifically as the underlying data gets harder and less GAN-clean, from losing on CTGAN to winning on the real, unaugmented data. What's still open: version 3 has only been verified on CTGAN at full depth (the SMOTE and original numbers above come from a lighter, single-configuration comparative pass, not the full ablation); its hyperparameters — landmark count, alignment subset size, qubit-pairing topology — were not systematically searched; and no version-3 equivalent exists yet for VQC. All results here were produced on a circuit simulator, not physical quantum hardware, so this is evidence of a useful quantum-inspired similarity measure on this dataset, not a claim of quantum computational speedup.

Appendix — Full Results Table

Best-seed accuracy for every model / version / dataset combination tested, against each configuration's own matched-feature classical baseline. “Goal met” means the quantum model matched or beat both classical models on identical inputs.

Model	Version	Dataset	Accuracy (%)	Classical SVM (%)	Classical RF (%)	Goal met?
QSVM	v1	CTGAN	45.0	66.0	68.7	No
QSVM	v1	SMOTE	35.0	44.0	45.3	No
QSVM	v1	Original	35.0	33.3	46.7	No
QSVM	v2	CTGAN	55.0	76.0	74.0	No
QSVM	v2	SMOTE	40.0	46.7	50.7	No
QSVM	v2	Original	47.5	46.7	51.3	Partial (beats SVM only)
QSVM	v3	CTGAN	87.5	80.7	87.3	Yes
QSVM	v3	SMOTE	60.0	52.7	50.7	Yes
QSVM	v3	Original	50.0	44.0	46.7	Yes
VQC	v1	CTGAN	45.3	66.0	68.7	No
VQC	v1	SMOTE	40.0	44.0	45.3	No
VQC	v1	Original	34.0	33.3	46.7	No
VQC	v2	CTGAN	43.3	76.0	74.0	No

Model	Version	Dataset	Accuracy (%)	Classical SVM (%)	Classical RF (%)	Goal met?
VQC	v2	SMOTE	35.3	46.7	50.7	No
VQC	v2	Original	39.3	46.7	51.3	No
VQC	v3	CTGAN	45.3	80.7	87.3	No
VQC	v3	SMOTE	34.0	52.7	50.7	No
VQC	v3	Original	40.0	44.0	46.7	No

5 Week 5

QTagger+ Week 5 Summary

Quantum-Augmented Malware Classification — Continuous Pipeline Narrative

Week 5 began with one unfinished thread from Week 3: the QGAN's synthetic malware data had only ever been produced by a broken 30-epoch training run, so every comparison downstream of it was provisional. The first job was to redo that run properly at 100 epochs, and once the corrected data was in hand, the week turned into a single continuous pipeline — the same corrected QGAN output was pushed through every test already built for SMOTE and CTGAN, one after another, so all three synthetic-data methods and the untouched original data could finally be judged on identical ground. Each test fed the next: the downstream-classifier run raised a question that the fidelity metrics answered differently than expected, which in turn was checked against the quantum classifier pipeline, which produced a headline result that then had to be stress-tested and, in the end, partly overturned. In parallel, a full week-long thread went into understanding why the QGAN itself was the weakest of the three generators in the first place, tracing that all the way down to a known failure mode in variational quantum circuits and running seven separate attempts to fix it. What follows is that whole chain, in the order it actually happened.

From a broken baseline to a fair four-way comparison

With the 100-epoch QGAN run corrected, the same four downstream classifiers already used for SMOTE and CTGAN — Random Forest, XGBoost, LightGBM, and SVM — were pointed at it, all scored against one shared, untouched, held-out test split. Before trusting any of the numbers, that fairness was checked directly rather than assumed: every classifier's “original-only” baseline F1 matched to six decimal places across all four runs, which meant whatever spread showed up next could only be coming from the augmentation method itself. Getting to that point required fixing a real problem first — a plain merge of real and synthetic rows broke on this dataset because of duplicate rows, so a hash-based, duplicate-safe split had to be built for SMOTE and CTGAN before any comparison could be trusted. That fix surfaced its own finding: SMOTE keeps every real row untouched and adds synthetic rows on top, while CTGAN's output is 100% synthetic — part of why the two behave so differently once a classifier is trained on them.

The results that came out the other side were not close. SMOTE produced clear, consistent gains everywhere — Random Forest moved from 0.8127 to 0.8988 (+8.6 F1 points), XGBoost from 0.8211 to 0.8681 (+4.7 points), LightGBM from 0.8129 to 0.8419 (+2.9 points). CTGAN went the other way, mildly negative on every classifier tested (XGBoost dropped from 0.8211 to 0.8083). The corrected QGAN numbers came in essentially flat against the original baseline (Random Forest 0.8121, XGBoost 0.8200, SVM 0.6039) — not because training had failed this time, but because the augmentation scheme only ever generates enough synthetic rows to reach 2–5% of the affected classes' training volume, too small a share to move a classifier's F1 in either direction.

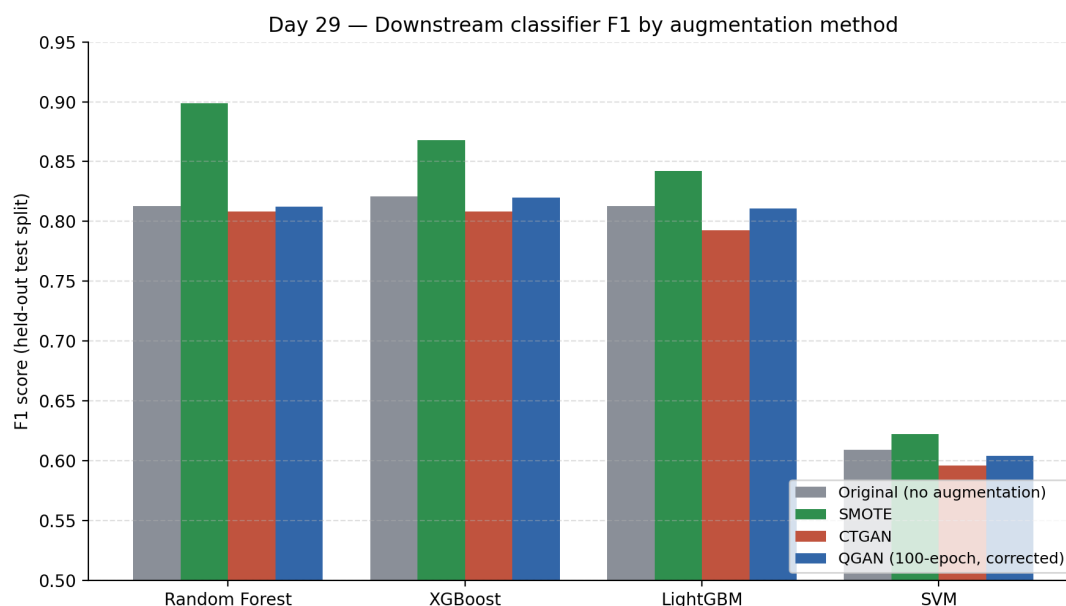


Figure 1. Downstream classifier F1 by augmentation method, all four evaluated on one shared held-out split. SMOTE is the only method with a consistent lift; CTGAN is mildly negative everywhere; the corrected QGAN run is flat by design, not by failure.

That downstream result immediately raised a second question: was QGAN just contributing too little volume to matter, or was its synthetic data actually a poor statistical match for the real distribution regardless of volume? To answer that, the KS statistic, Wasserstein distance, and MMD were computed for all four methods, per malware class, using one identical scoring function so the comparison would again be apples-to-apples. The ranking that came out was completely clean and held on every single class and every single metric: SMOTE beat CTGAN, and CTGAN beat QGAN, by wide margins. On Ransomware, SMOTE's KS_median was 0.012 against CTGAN's 0.091 and QGAN's 0.328 — roughly a 10x gap between SMOTE and CTGAN, then a further 3x gap down to QGAN. Trojan showed the same pattern (0.013 / 0.112 / 0.293); on Spyware, QGAN generated no synthetic rows at all, since the real count already met target, leaving SMOTE (0.010) well ahead of CTGAN (0.133).

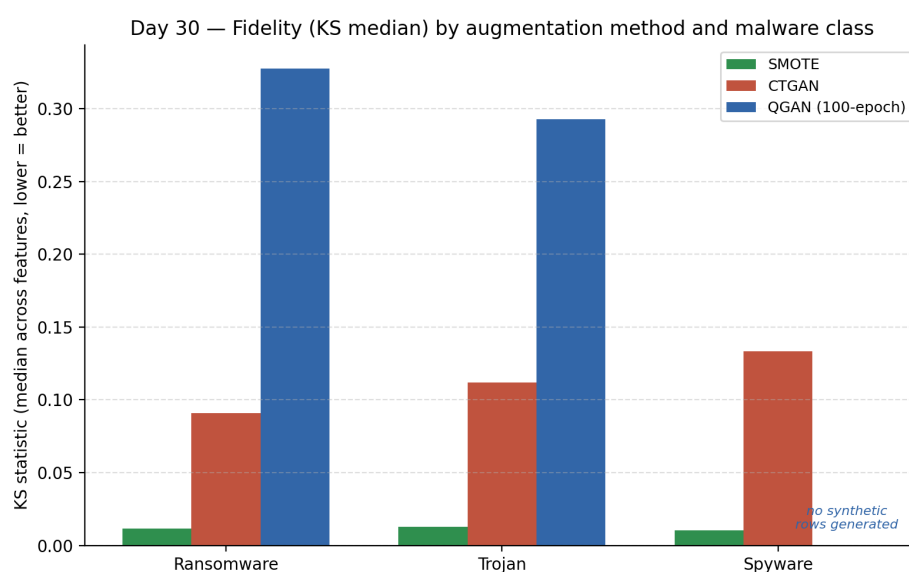


Figure 2. KS-statistic fidelity (median across features, lower is better) by method and class. SMOTE sits roughly an order of magnitude closer to the real distribution than CTGAN, which in turn sits roughly

three times closer than QGAN.

Putting the two results side by side produced the finding that shaped how the rest of the week was read: fidelity rank does not predict downstream impact. CTGAN beat QGAN cleanly on every fidelity metric, yet lost to it on downstream classifier impact — CTGAN was mildly negative, QGAN was flat rather than negative. Looking statistically close to real data and actually helping a classifier learn turned out to be two different properties of synthetic data, not one proxy for the other.

Carrying the corrected data into the quantum classifier pipeline

With downstream impact and fidelity both settled, the corrected QGAN data was carried one step further, into the same V3 QSVM/VQC architecture already used for CTGAN and SMOTE — identical code, only the input swapped, to keep the test fair. The first run was caught before it could be trusted: QGAN's as-uploaded data was only about 2–5% synthetic against CTGAN's roughly 33–40%, so QGAN's data was rebuilt at matched scale and re-run. With that corrected, all three QSVM architecture versions were tested at a single seed: V1 scored 0.350 against a classical best of 0.567, V2 scored 0.550 against 0.633, and V3 scored 0.625 against 0.587 — the only version where QGAN's single-seed number beat its classical counterpart.

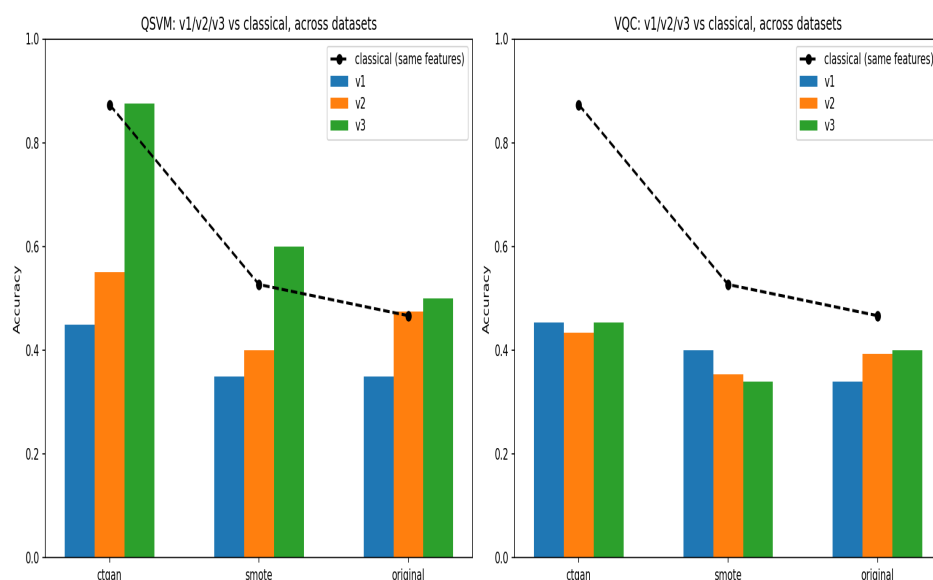


Figure 3. QSVM/VQC pipeline versions 1–3 against their classical baselines (single-seed exploratory comparison, before the multi-seed correction below).

That 0.625 result, and the more striking 0.875 CTGAN result sitting alongside it from earlier in the project, did not survive closer scrutiny — and the way they were checked is worth keeping on its own merits. An uploaded 18-config comprehensive study was independently verified by matching its CTGAN seed = 7 result to this project's own run to four decimal places, and that verified code was then used to expand every dataset from one or two seeds to three-to-five. The expansion overturned the original headline entirely: CTGAN's reported 0.875 collapsed to a mean of 0.708 ± 0.170 with only a 1-in-3 win rate against classical models; SMOTE's 0.600 became 0.444 ± 0.094 (1-in-4); QGAN's 0.625 became 0.540 ± 0.049 (1-in-5). Only the original, unaugmented data held its ground — 0.542 ± 0.031 , a 3-out-of-3 win rate, and by far the lowest variance of the four datasets tested.

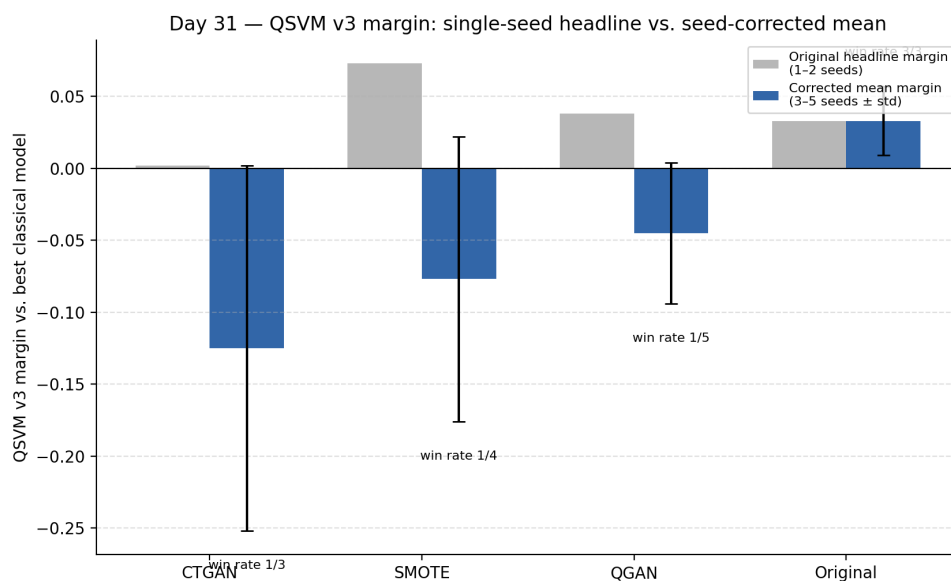


Figure 4. QSVM v3 margin over the best classical model: the original single/best-of-two-seed headline (grey) versus the corrected multi-seed mean $\pm$ std (blue). Only Original data keeps a positive, low-variance margin once more seeds are sampled.

So the corrected picture is the opposite of the first-pass headline: QGAN loses to classical models at every architecture version once properly seed-tested, and it is only the original, unprocessed, unaugmented data that shows a robust, low-variance quantum advantage. That correction was then checked against an entirely separate piece of evidence — a pre-existing 27-config V2 grid, run independently and earlier in the project, which had already found the same shape of result (Original 56% quantum win rate, SMOTE 33%, CTGAN 11%). Two different pipeline versions, two different seed sets, and the same conclusion both times: quantum's edge shows up reliably only on harder, messier, unaugmented data, and not on any of the three synthetic-data methods tested, regardless of which one generated the data.

Comparison axis	Best method	Worst method	Consistent across...
Fidelity (KS / Wasserstein / MMD)	SMOTE	QGAN	every class, every metric
Downstream classifier F1	SMOTE (clear lift)	CTGAN (mildly negative)	all 4 classifiers
Quantum vs. classical (QSVM v3, corrected)	Original data (3/3 win rate)	CTGAN (1/3 win rate)	V2 grid and corrected V3, independently

Table 1. The three axes of comparison, consolidated once all three threads above were complete. None of the three ranks the four methods the same way.

Why the QGAN kept losing: diagnosing discriminator dominance and the barren-plateau wall

Running QGAN's data through every one of the comparisons above only sharpened the question the fidelity numbers had already raised: why was the QGAN's synthetic data so much further from the real distribution than either CTGAN's or SMOTE's in the first place? Answering that meant going back to the QGAN's own training process rather than its output, using the full per-batch loss history saved across all 7,400–7,800 batch-steps per class over the full 100-epoch run, not just the periodic epoch printouts most training logs keep. All three malware classes showed the identical failure signature: the discriminator's loss fell steadily while the generator's loss climbed, and the gap between the two widened and grew noisier as training went on instead of settling into a stable equilibrium. Ransomware's discriminator loss

fell from 1.356 to 1.136 while its generator loss rose from 0.837 to 1.140, with the second half of training roughly 1.8x noisier than the first; Trojan and Spyware showed the same shape, with Spyware's oscillation nearly tripling by the end of the run.

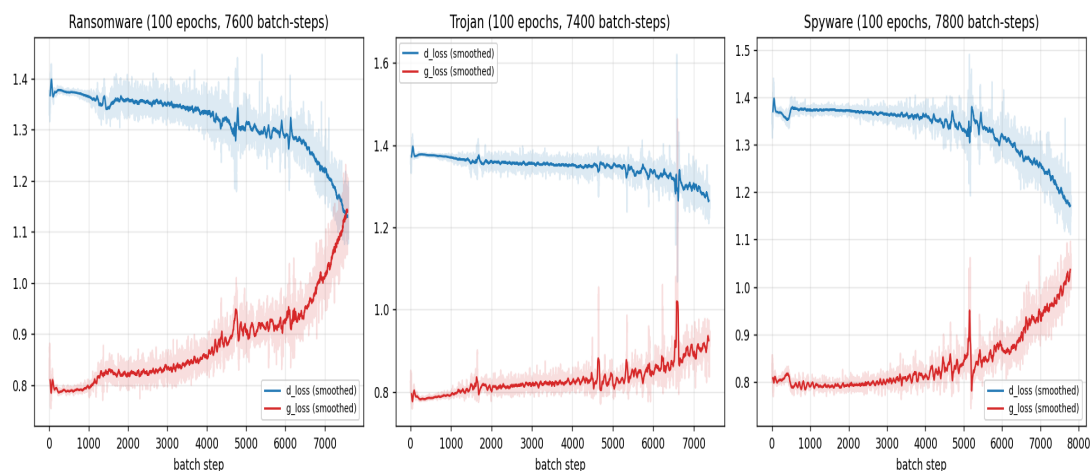


Figure 5. Discriminator loss (falling) versus generator loss (rising) across all three malware classes, full 100-epoch baseline run. The gap widens and gets noisier in every class instead of converging — the signature of discriminator dominance.

The obvious first suspect was a learning-rate imbalance, and it was ruled out directly rather than assumed away: the generator's learning rate ($7e-3$) was already 3.5x higher than the discriminator's ($2e-3$), and both networks received exactly one update per batch, a confirmed 1:1 ratio in the training loop. So the generator already had a nominal training-speed advantage over its adversary and still lost consistently. That ruled-out explanation pointed toward something structural instead: a representational capacity mismatch. The generator was a 4-layer variational quantum circuit built on 6 qubits, whose expressive power is bounded by circuit depth and qubit count rather than by a freely scalable parameter count the way a classical network's is. The discriminator, by contrast, was a classical multilayer perceptron with only 16 and 8 hidden units in its two layers — small by classical standards, but still comfortably more than enough capacity for its own sub-task, which was just binary real-versus-fake classification, a fundamentally easier problem than reproducing an entire data distribution from noise.

Increasing generator depth: the direct test of the capacity diagnosis, and the barren-plateau wall it hit

Once the diagnosis pointed at generator capacity, the most direct-sounding fix was also the most obvious one: give the generator more capacity by adding more variational layers. That instinct runs into a well-documented limitation of variational quantum circuits known as a barren plateau. As a variational circuit gets deeper or wider, under fairly general conditions the gradients used to train it shrink exponentially with the number of qubits involved — not gradually, but to the point where the optimizer effectively receives no usable signal at all. The practical consequence is counter-intuitive: a deeper circuit is not automatically easier to train just because it has more nominal expressive capacity on paper. The added depth can make the loss landscape flatter and harder to navigate at exactly the same time it makes the circuit theoretically more expressive, and for gradient-based training, a flat landscape with no gradient to follow is worse than a slightly-too-small circuit that can still be trained.

This is exactly what was tested, and exactly what happened. The generator's depth was increased from 4 layers to 6, isolated to a single class (Trojan) and retrained from scratch so the comparison against the untouched 4-layer checkpoints would be clean. The result was the direct opposite of the hypothesis, and by a wide margin: the discriminator/generator loss gap ended up larger, not smaller, than the original 4-layer run, and fidelity (KS_median) got worse — moving from 0.216 to 0.254 in the PCA-encoding

space, with the Wasserstein distance increasing by roughly 74%. In other words, giving the generator more qubits and layers did not close the capacity gap — it pushed the generator into a barren-plateau regime where its gradients became noisier and less useful, and training got measurably worse on every measure tracked. That single negative result matters more than a positive one would have here: it is direct, first-hand evidence that this specific circuit family cannot be fixed simply by adding depth, and it ruled out the single most intuitive fix before any further time was spent on it.

What was tried, in the order it was tried

With the most direct capacity-side fix ruled out, five things had already been checked by the point the diagnosis was complete: the full per-batch loss history had been pulled and inspected class by class; the learning-rate imbalance had been checked and ruled out; the generator's depth had been increased from 4 to 6 layers and found to make training worse, consistent with a barren plateau; a cheap post-hoc output-bias calibration had been tried, matching the synthetic data's mean and standard deviation to the real data's after generation; and quantile mapping — remapping each synthetic feature to the real data's empirical rank distribution — had been tried as a stronger, non-parametric alternative.

The output-bias calibration did not help: the moment-matching loss already active during training had already closed most of the first- and second-moment gap on its own, and forcing it to exactly zero through a post-hoc affine correction distorted the shape of the distribution instead, making fidelity worse on two of the three classes tested.

Quantile mapping: what it was meant to fix, what it actually produced, and why it was thrown out

Quantile mapping was tried next as a stronger, non-parametric alternative to the output-bias calibration above, and it is worth walking through in full because the gap between what it promised and what it actually delivered is itself an important, honestly-reported result. The KS statistic and Wasserstein distance — the two fidelity metrics used everywhere else in this project — are both, by construction, direct comparisons of distribution shape between a real sample and a synthetic one. The hypothesis behind quantile mapping was to target that shape comparison as directly as possible rather than only matching the first two moments (mean and standard deviation) the way the output-bias calibration had: for every one of the 54 real-valued features, per malware class, each synthetic value would be re-mapped onto the rank position of the corresponding real value under an empirical cumulative distribution function (CDF) built from the same held-out real sample used for scoring.

What was actually implemented matched that description exactly — a per-feature empirical-CDF rank remap, applied after generation, using the held-out real sample as the reference distribution for every class. What it produced looked, on paper, like the best result of the entire stabilization effort by a wide margin. Across all three malware classes, the KS_median statistic collapsed from a baseline of roughly 0.25–0.28 down to about 0.0001 — effectively zero — and the Wasserstein distance fell in the same way, from roughly 0.6–0.9 down to about 0.004–0.009. Ransomware's KS_median moved from 0.2808 to 0.0001, Trojan's from 0.2467 to 0.0001, and Spyware's from 0.2685 to 0.0001, with Wasserstein distance falling by two to three orders of magnitude alongside it on every class.

Quantile mapping drives fidelity metrics to near-zero — invalidated as evaluation leakage, not a genuine generator improvement

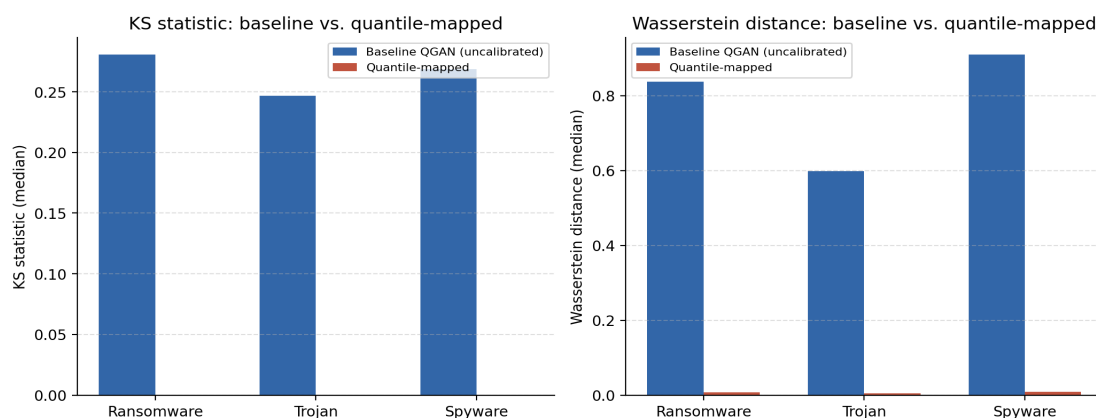


Figure 6. KS statistic and Wasserstein distance, baseline QGAN versus quantile-mapped output, all three malware classes. The near-total collapse to zero is not a sign the generator improved — it is the signature of the evaluation metric being answered with its own reference data.

That result was not reported as a win, and the reason is the same reason it looks too good in the first place: with equal real and synthetic sample counts, remapping every synthetic value onto the rank position of a real value forces the synthetic feature's marginal distribution to become a near-exact copy of the real data it is being scored against. The generator itself produced nothing different — the transformation was applied entirely after generation, using the evaluation set as its own reference, so the KS test was effectively comparing the real held-out sample to a rank-sorted copy of itself. This is evaluation leakage: the metric can be driven to its best possible value by a purely post-hoc bookkeeping step that has nothing to do with whether the underlying generator can produce new, realistic, previously-unseen samples, which is the actual property the fidelity metrics are meant to stand in for. Because of that, the quantile-mapped numbers were explicitly excluded from every other table, chart, and comparison used across the rest of the project — including the master comparison table and every downstream and fidelity result discussed earlier in this summary — and the attempt is documented here specifically as a negative methodological lesson: a metric that can be forced to its optimum by leakage is not, on its own, sufficient evidence that a generator has actually improved.

With the capacity-side fix (more qubits/layers) ruled out by the barren-plateau result, attention shifted to training-dynamics fixes that work around the capacity mismatch instead of trying to close it directly. Three further directions were identified and tested to completion: weakening the discriminator so its task became relatively harder — through dropout, one-sided label smoothing, and a smaller network; rescheduling the learning rate so it could change independently per circuit layer rather than globally; and the two-timescale update rule (TTUR), which gives the generator and discriminator genuinely different learning rates rather than a single shared schedule, to compensate directly for the generator's structural disadvantage.

Removing label smoothing turned out to be the single strongest result of the whole stabilization effort. The baseline configuration already softened the discriminator's target for real samples from 1.0 to 0.9 as a standard stabilization technique — but removing that softening entirely, and testing the counter-intuitive hypothesis that it was giving an already-dominant discriminator an even easier target, produced a clear, fully-confirmed improvement on both classes tested: Ransomware's KS_median improved from 0.328 to 0.253 (about 23% better) and Trojan's from 0.293 to 0.274 (about 6.5% better), with the discriminator/generator loss gap narrowing on both classes rather than widening. Shrinking the discriminator's hidden layers from 16+8 down to 8+4 units gave a mixed result: the loss gap narrowed on both classes, matching the same qualitative direction as label-smoothing removal, but fidelity only improved on Ransomware (0.297) while Trojan regressed slightly (0.308) — a useful negative-adjacent finding on its

own, since it shows a narrowing loss gap does not guarantee improved fidelity on every class. Widening the generator-to-discriminator learning-rate ratio through TTUR gave a modest, confirmed gain at 10x (Ransomware 0.314, Trojan 0.263) without resolving the underlying dominance pattern, and a rough 30x check showed no further improvement, flat-to-worse on Ransomware specifically. A per-layer generator learning-rate schedule, giving each of the four variational layers its own decay rate, was tested as a fast, short-epoch check and came back inconclusive, and also surfaced a real, non-fatal checkpoint-saving bug that needs fixing before that variant can be trusted for a full run.

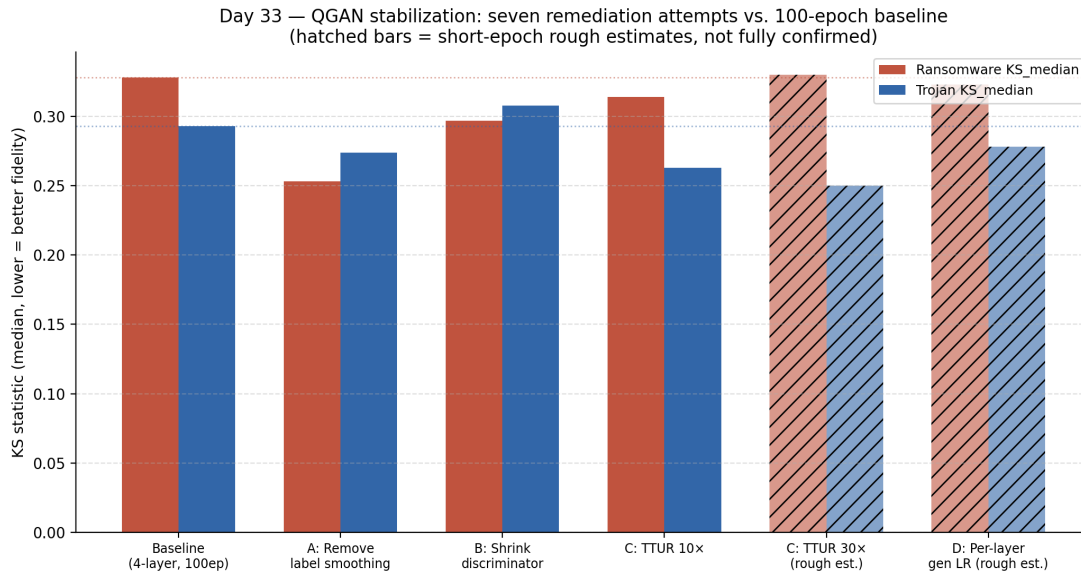


Figure 7. All seven remediation attempts against the 100-epoch baseline. Removing label smoothing is the only fully-confirmed improvement on both classes; hatched bars are short-epoch rough estimates, not full 100-epoch confirmations.

Intervention	Ransomware KS_median	Trojan KS_median	Verdict
Baseline (no fix)	0.328	0.293	reference
Verify per-batch loss history	—	—	diagnosis step — confirmed
Check learning-rate imbalance	—	—	discriminator dominance ruled out as the cause
Increase generator depth (4→6 layers)	—	0.254 (PCA space)*	did not work — barren-plateau signature
Output-bias calibration	worse (2/3 classes)	—	did not work
Quantile mapping	≈ 0 (invalid)	≈ 0 (invalid)	did not work — evaluation leakage
Remove label smoothing	0.253 (−23%)	0.274 (−6.5%)	worked — strongest confirmed result
Shrink discriminator	0.297 (−9%)	0.308 (+5%, worse)	partial
TTUR, 10× ratio	0.314 (−4%)	0.263 (−10%)	partial, confirmed
TTUR, 30× ratio (rough est.)	≈ 0.33 (flat)	≈ 0.25 (rough)	did not clearly improve on 10×

Intervention	Ransomware KS_median	Trojan KS_median	Verdict
Per-layer generator LR schedule (rough est.)	≈ 0.32 (flat)	≈ 0.28 (−5%)	inconclusive; checkpoint bug found

*Table 2. Every diagnosis step and remediation attempt, in the order it was run. *Capacity-increase figures were measured in the PCA-encoding space and are not on the same numerical scale as the original-feature-space baseline. None of the round-two variants has been adopted as the project's official QGAN checkpoint — the downstream and fidelity numbers used everywhere earlier in this summary still reflect the original, unmodified generator.*

Put together, four of the seven things tried gave no benefit or made fidelity worse — including the generator-depth increase, which is itself useful evidence precisely because it directly tested the capacity-mismatch diagnosis and hit the barren-plateau wall predicted by it. Two gave partial, mixed, or unconfirmed improvements. One — removing label smoothing — gave a clear, fully confirmed improvement on both evaluated classes without resolving the underlying dominance pattern: even under that best-performing fix, the generator's own loss still rose over training, meaning the discriminator still ended up ahead, just less overwhelmingly than in the untreated baseline. The state of the QGAN at the end of Week 5 is therefore: partially stabilized through training-dynamics interventions, with the deeper architectural capacity mismatch — and the barren-plateau ceiling on the most direct fix for it — still an open problem.

Where the week left off

The last two days of the week were spent making sure everything above actually held together. The project's best single quantum-classifier number — 0.875 accuracy on CTGAN using the V3 pipeline's XGBoost feature selection at 8 qubits, seed 7 — was verified directly against its raw run data, saved model file, and ablation results rather than taken on faith, and the ablation confirmed both of V3's components were necessary: a local kernel alone reached 0.825, KTA alignment alone reached 0.725, and the two combined reached the full 0.875. That headline was explicitly caveated against the same seed-correction finding from earlier in the week — the independently-run 27-config V2 grid had already found CTGAN to be the worst dataset for quantum overall, at a 1-in-9 win rate — so the 87.5% number is real and reproducible from the raw data, but represents one favorable configuration rather than a general result.

Everything was then packaged into one consolidated repository, and a final consistency check confirmed that the newly-completed Original baseline matched QGAN's originally-computed baseline to six decimal places, verifying that an identical test split had held across separate work sessions run at different times. No open items remained by the end of the week. The state the pipeline was left in: SMOTE wins clearly on both fidelity and downstream classifier impact; QGAN and CTGAN both trail it, for different reasons and in different ways; quantum shows a real, low-variance advantage only on the original, unaugmented, harder data, and not on any synthetic-data method tested; and the QGAN's own instability is understood down to its root cause, partially treatable through training-dynamics fixes, and still blocked at the architectural level by the barren-plateau limitation on directly increasing generator capacity.